# Adaptive Auto-Scaling with Dynamic Step Size Using AWS Lambda and AWS CloudWatch

Valerio Mesiti, *1936543, Sapienza University of Rome,*
Gabriele Moretti, *1958932, Sapienza University of Rome,*
Niccolo Siciliano, *1958541, Sapienza University of Rome*

**Abstract**—This project presents the implementation of an adaptive auto-scaling strategy for cloud environments, based on the algorithm proposed by Netto et al. [1] The approach leverages AWS Lambda and Amazon CloudWatch to dynamically monitor resource utilization and scale EC2 instances in response to fluctuating workloads.
The goal is to evaluate the effectiveness of the adaptive step sizing mechanism in providing scalability and efficient resource utilization under variable load conditions. Performance tests were conducted considering three distinct workloads, inspired by realistic traffic models: bursty workload, clustered workload, and peaky workload. The analysis was based on the observation of two key metrics, CPU utilization and desired capacity over time. The results of the experiments demonstrate that the system can adapt responsively, which confirms the suitability of the algorithm in real-world situations.

**Index Terms**—Cloud Computing, Auto-scaling, Adaptive Step Size, AWS Lambda, EC2, CloudWatch, Performance Evaluation, Scalability.

## 1 Introduction

In this project, conducted for the Cloud Computing course, we implemented an adaptive auto scaling algorithm for cloud environments, inspired by the work of Netto et al. The goal of the algorithm is to dynamically adjust the computational capacity of the system, increasing or decreasing the number of EC2 instances based on the actual workload, to ensure efficient use of resources.

To monitor resource usage in real time and scale EC2 instances in response to different workloads, the adopted approach leverages the integration between AWS Lambda and Amazon CloudWatch. The project focuses on the effectiveness of the algorithm in adapting reactively and efficiently to load fluctuations, with particular attention to scalability and optimal use of server-side resources.

## 2 Architecture and implementation

The architecture of the implemented system is based on four main components:

1) An Auto Scaling Group (ASG) of EC2 instances
2) An Application Load Balancer (ALB)
3) An AWS Lambda function that implements the adaptive scaling algorithm
4) The Amazon CloudWatch service, used both to collect metrics and to trigger scaling actions.

**Auto Scaling Group** The Auto-Scaling Group (ASG) is a set of computing resources that automatically adjusts to demand, increasing or decreasing the number of instances based on the load. It continuously monitors the average CPUUtilization of its instances and adjusts the capacity to maintain performance and cost efficiency. In our experiment, the ASG was configured with a minimum desired capacity of 1 instance and a maximum of 15 instances, ensuring elasticity while preventing over-provisioning.

**Application Load Balancer** The Application Load Balancer (ALB) distributes incoming requests across the instances in the Auto Scaling Group. It ensures that only healthy InService instances receive traffic, thereby improving fault tolerance and availability. The ALB follows a round-robin distribution strategy, cyclically redirecting requests to different instances. This setup facilitated performance testing from remote clients, since the ALB acted as a single entry point to the dynamic group of instances.

**AWS Lambda** The AWS Lambda function implements the adaptive scaling algorithm. It is written in Python and is responsible for adjusting the desired capacity of the ASG. Unlike time-based scheduling, Lambda is invoked through a system of Amazon CloudWatch Alarms. In our experiment, two alarms were defined:

- A scale-out alarm, triggered when the average CPUUtilization of the ASG exceeds 80%,
- A scale-in alarm, triggered when utilization drops below 45%.

When either alarm is triggered, it sends a notification via Amazon SNS (Simple Notification Service), which in turn invokes the Lambda function. The function then updates the desired ASG capacity accordingly. This approach allows for a band-threshold control mechanism, avoiding frequent oscillations and ensuring stable scaling decisions.

**Amazon CloudWatch service** Amazon CloudWatch is the monitoring backbone of the system. Collects detailed metrics on CPUUtilization, instance health, and scaling actions. Beyond monitoring, CloudWatch enables alarm-based automation, which links metrics to Lambda invocations

*Manuscript delivered on 08/31/2025*

through SNS notifications. In this setup, CloudWatch not only provided visibility into system performance, but also served as the control signal generator for the adaptive scaling process.

### 2.1 Description of the implemented algorithm

The implemented algorithm is a Lambda function that executes adaptive automatic scaling based on the model proposed in the paper "Evaluating Auto-scaling Strategies for Cloud Computing Environments" (Netto et al., 2014). This approach uses a reactive strategy with an adaptive step size to dynamically resize an Auto Scaling Group (ASG) based on CPU usage.

The goal of the algorithm is to maintain the average usage of EC2 instances within a defined range [L, U], by adjusting the number of active instances mt through a dynamically calculated step st, to minimize the Auto-scaling Demand Index (ADI), a metric that sums the distances between the actual use of resources and the upper and lower limits of the target range in each time interval. An ADI value close to zero indicates effective resource optimization, minimizing both the risk of overload and the risk of under-utilization.

### 2.2 Functioning of the algorithm

The function reads the average CPU usage for all active instances in the ASG group using Amazon CloudWatch. Depending on whether the current use is:

- Within the interval [L, U], no scaling is performed.
- Greater than U, a scale-out (addition of instances) is performed.
- Lower than L, a scale-in (removal of instances) is performed.

The number of instances to add or remove (step size st) is calculated according to the formulas from the paper:

- For scale-out
  - Lower bound Lt = mt × [(ut - U) / U]
  - Upper bound Ut = mt × [(ut - L) / L]
  - Step size st = alpha × Lt + (1 - alpha) × Ut
- For scale in
  - Lower bound Lt = mt × [(U - ut) / U]
  - Upper bound Ut = mt × [(L - ut) / L]
  - Step size st = (1 - alpha) × Lt + alpha × Ut

Where alpha [0,1] is the configurable aggression parameter, lower values mean less aggressive scaling. The step size is then converted to an integer and is limited within a certain minimum and maximum range.

When scaling is required, the function changes the DesiredCapacity of the Auto-Scaling Group (ASG) via boto3, however, this change is done immediately without waiting for the standard cooldowns as the adaptive strategy still considers them.

The algorithm accurately reflects the adaptive strategy with the aggressiveness depicted in the paper, enabling a more flexible approach to changes in load, aids in minimizing the number of unnecessary scaling actions, and thus keeps the system near the target usage level, further benefiting the service quality and cost reduction.

## 3 Testing Methodology

The objective of the experimental evaluation is to assess the dynamic behavior of the system under varying workload conditions. Specifically, the analysis aims to verify whether the implemented adaptive algorithm can effectively adjust the number of EC2 instances (scale-out and scale-in) based on CPUUtilization, maintaining performance close to a predefined target.

The tests follow an empirical evaluation approach. The workload was generated using the stress tool, which was automatically installed and executed at the start of the instance through a user data script embedded in the EC2 launch template. This ensures that newly launched instances contribute to the global system load from the moment they become active.

Each experiment lasted approximately 3-4 hours. The main metrics analyzed are the following:

- CPUUtilization of EC2 instances
- GroupDesiredCapacity of the Auto Scaling Group

We performed a total of three experiments, varying the intensity and duration of the load to evaluate the reactivity and stability of the control mechanism. The metrics were collected using Amazon CloudWatch and exported in CSV format for analysis with tools such as Python matplotlib.

### 3.1 Purpose and Scope of the Evaluation

The purpose of performance evaluation is to check if and to what extent the adaptive variable-step auto-scaling algorithm, implemented via AWS Lambda and AWS CloudWatch, is able to dynamically adjust the number of EC2 instances in an Auto-Scaling Group (ASG) in response to variable and continuous load conditions.

The analysis focuses on the automatic scalability of the system: in particular, it observes whether the algorithm is able to react effectively to increasing or decreasing loads, maintain CPU utilization within desired thresholds [L,U], and adapt the number of instances in a proportional and smooth manner (i.e., avoiding under/over-provisioning).

The evaluation scope is systemic (server-side): the focus is on the behavior of the system from the service provider's perspective, not the end users. Metrics such as:

1) Average CPU usage of EC2 instances.
2) Number of active instances over time.

### 3.2 Features/Aspects of the Cloud Services to be Evaluated

The evaluation focuses on two main aspects of the AWS cloud services involved in the auto-scaling mechanism:

- Behavior of the Auto Scaling Group (ASG). The Auto Scaling Group is the component responsible for the automatic addition and removal of EC2 instances. The aspects evaluated include:
  - how quickly the group scales in response to a change in load
  - if the number of active instances is consistent with the step size dynamically calculated by the algorithm;

  - if the system avoids oscillations or bounces (too frequent or unstable scaling up and down).

- Performance of EC2 instances. EC2 instances represent the computational resources allocated in response to load. The measured aspects include:
  - Average CPU usage, to check if the system keeps usage within the desired range [L,U];
  - Load distribution among the instances, to assess whether the Application Load Balancer (ALB) and the ASG are correctly balancing the traffic;
  - Time required for a new EC2 instance to be launched and become "healthy" in the Target Group (an indirect indicator of the effectiveness of automatic provisioning).

Since the algorithm uses a variable step strategy, particular attention is paid to:

- how the step size varies based on the load;
- whether the system scales more aggressively under high loads and more gradually under light loads;
- the effect of the aggressiveness parameter setting () on the system's behavior.

### 3.3 Performance Metrics Analyzed

Performance evaluation is system-oriented and focuses on the infrastructure's ability to adapt to variable loads while ensuring acceptable performance. The selected metrics are derived from AWS's standard namespaces and have been chosen to objectively assess the system's behavior over time, in relation to throughput, resource utilization, scalability, and availability. The metrics used are as follows:

- AWS/EC2 $\rightarrow$ CPUUtilization, measures the percentage of CPU usage for each EC2 instance. Indicates the level of computational load of the resources. It is also the benchmark metric for scaling decisions of the implemented algorithm.
- AWS/AutoScaling $\rightarrow$ GroupDesiredCapacity, measures the number of EC2 instances that the Auto-Scaling group intends to keep active at any given time. It is the main metric for analyzing the dynamic scalability behavior of the system.

These metrics have been selected because they cover the main aspects of performance at the system level and are suitable for objectively describing how the infrastructure behaves under load.

### 3.4 Tool for Evaluating Performance

The infrastructure for this project is based on EC2 instances managed by an Auto-Scaling Group and balanced through an Application Load Balancer. This architecture allows the use of lightweight, fully integrated AWS tools for monitoring and performance evaluation, without relying on external software.

The selected tools enable the collection of key metrics related to throughput, availability, scalability, and resource utilization, providing a system-oriented assessment of the auto scaling mechanism. In particular:

- Amazon CloudWatch: serves as the primary monitoring tool, collecting metrics from the Application Load Balancer, EC2 instances, and the Auto Scaling Group. CloudWatch provides detailed insights into system performance while remaining entirely within the AWS ecosystem.
- User-data scripts on EC2 instances: each instance runs a simple HTTP server that receives requests from a remote client. Each request includes parameters for the stress -ng command, allowing controlled CPU stress tests by specifying the number of CPU cores and the load percentage for each core. This approach ensures repeatable and consistent testing across instances.

By combining CloudWatch monitoring with programmable stress testing, the system allows for a comprehensive, realistic, and reproducible evaluation of the auto-scaling behavior, fully integrated within AWS.

### 3.5 Design of the performance evaluation experiments

The experiments involve sending HTTP POST requests to the /start -stress endpoint of the Application Load Balancer (ALB). Each request triggers the execution of the stress -ng tool on an EC2 instance, which generates CPU-intensive load according to the parameters specified (number of CPUs, percentage of load, and execution timeout).

To evaluate system behavior under different conditions, three distinct workload profiles were designed, inspired by realistic traffic patterns:

- Bursty workload: characterized by alternating phases of low activity (warmup and cooldown) and sudden bursts of high-intensity requests. This workload simulates applications with unpredictable spikes in demand.
- Clustered workload: requests are grouped into large clusters of high activity, separated by cooldown periods. This reflects scenarios such as batch processing or synchronized user interactions.
- Peaky workload: the system is subject to sharp peaks of increasing intensity, where requests progressively stress more CPUs and higher load percentages. This simulates severe and short-lived demand surges.

Each workload is implemented through a bash script that controls request frequency, payload parameters, and cooldown intervals, ensuring reproducibility. The intensity and timing of the stress phases differ across the three profiles, but all are designed to test the elasticity of the Auto-Scaling Group under sudden or irregular CPU demands.

The experiments lasted approximately 3-4 hours per workload for a total of 225 requests per workload, during which the system remained fully operational and monitored through Amazon CloudWatch. To ensure fairness, the execution environment was kept constant across all runs: same VPC configuration, EC2 instance type, scaling policies, and metric collection methods.

This approach provides a realistic, controlled and repeatable testing framework, fully integrated into the AWS

ecosystem, to assess the responsiveness and efficiency of the adaptive scaling mechanism.

| Workload | Pattern | CPU Load (stress -ng params) | Request Phases | Purpose |
|---|---|---|---|---|
| **Bursty** | Alternating warmup, sudden bursts, cooldowns | 1–2 CPUs, 30–100% load, 200–250s | Multiple bursts separated by cooldown intervals | Simulates unpredictable traffic spikes (e.g., flash crowds) |
| **Clustered** | Large clusters of requests with cooldowns in between | 1–2 CPUs, 50–100% load, 200–250s | 3–4 main clusters, separated by long cooldowns | Represents batch-like workloads or synchronized user actions |
| **Peaky** | Sharp peaks of increasing intensity | 1–2 CPUs, 60–100% load, 150–200s | Progressive peaks with growing stress intensity | Tests system elasticity under extreme short-lived surges |

### 3.6 Experimental environment

The experimental environment has been set up within the AWS Academy Learner Lab platform to isolate the system under test and ensure full control of the resources.

The system is distributed over a single Virtual Private Cloud (VPC) and includes the following components:

- An Application Load Balancer (ALB), for distributing HTTP requests to EC2 instances;
- An Auto Scaling Group (ASG), for the automatic management of the number of EC2 instances based on CPU usage;
- A Launch Template to define the configuration of EC2 instances (type, AMI, user data, security group);
- An AWS Lambda function invoked periodically via Amazon CloudWatch Events, responsible for implementing adaptive scaling logic;

All resources have been allocated within the same availability zone to reduce internal latency and ensure consistency in testing. The environment configuration remained unchanged throughout the duration of the experiments, except for the applied load, to ensure that the observed variations can be attributed solely to the load dynamics and not to infrastructural changes. This configuration allows for controlled, repeatable tests that align with the goal of system-oriented performance evaluation.

### 3.7 Run performance evaluation experiments

The necessary metrics for the evaluation were collected through Amazon CloudWatch, using the tools available in AWS CLI to export datapoints with 1 minute granularity. The selected metrics correspond to those identified in Section 3.3 for system-oriented analysis.

For each experiment, the average value of the data points collected for each metric was calculated. The reference thresholds have been defined on the basis of realistic values for auto-scaling web systems. In particular, the operational target threshold of CPUUtilization is between 45% and 80%.

## 4 Results and Analysis

This section presents the performance results obtained during the experiments. The collected data are shown through time series plots, which highlight how the system reacted to different workload phases.

### 4.1 Workload 1 (Bursty)

The graph of CPU utilization with workload 1 (bursty) highlights alternating phases of low activity (heating and cooling) and bursts of high-intensity requests.


Fig. 1. Bursty CPU usage

Different episodes of load are observed, characterized by rapid increases up to 100% and fluctuations between high and low values. During the active phases, CPU utilization predominantly ranges between 75% and 100%, demonstrating efficient resource use. The ability to rapidly shift from low to high values indicates good system responsiveness to sudden changes in load. The regularity of the pattern across different episodes indicates stability and predictable behavior under this type of intermittent load.

The Desired Capacity graph highlights four scaling episodes that follow the phases of bursty load.

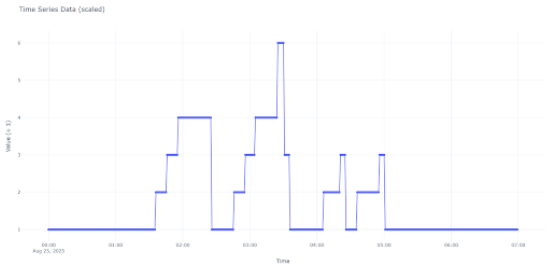


Fig. 2. Bursty desired capacity

The system always starts from a minimum configuration of 1 machine and dynamically scales up to a maximum of 6, maintaining stable intermediate configurations based on the intensity of the peaks.

After each active phase, it returns to the minimum configuration, optimizing costs during idle periods. The duration of the plateaus shows the stability of decision making, avoiding excessive fluctuations. In general, the behavior demonstrates a good balance between response to peaks and economic efficiency.

The value of the ADI calculated for this workload is equivalent to 9.524011200544498.

### 4.2 Workload 5 (Clustered)

The CPU utilization graph with workload 5 (clustered) shows a relatively stable trend, with values ranging between 50% and 78% and moderate fluctuations, consistent with the gradual nature of the load.

However, two anomalies are observed: a drop in 39% with a rapid recovery and a more critical collapse down

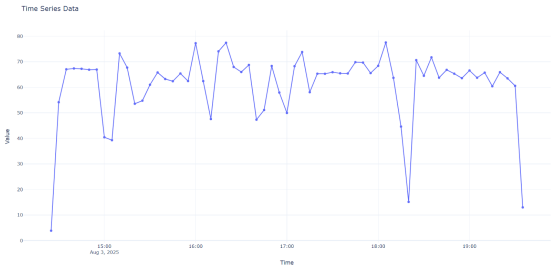


Fig. 3. Clustered CPU usage

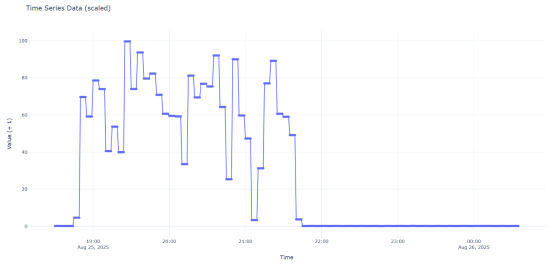


Fig. 5. Peaky CPU usage

to 15%, followed by instability and a further minimum in 13%. These significant drops indicate system protection mechanisms (throttling), resource saturation leading to performance degradation, or load management issues.

Overall, the system demonstrates good load management capabilities during most of the test, maintaining sustainable CPU utilization.

The Desired Capacity graph shows a progressive autoscaling behavior, characterized by four main levels: initial ( 1 machine), low intermediate ( 3), high intermediate ( 5-7), and maximum ( 9.5).


Fig. 4. Clustered desired capacity

Transitions occur gradually with prolonged plateaus, indicating operational stability and effective balance between capacity and demand. This trend, in stark contrast to bursty loads, reflects a progressive and well-calibrated management, optimized for the clustered nature of the workload.

The gradual trend aligns perfectly with the nature of the clustered workload. The system effectively anticipates and adapts to the increasing demand for resources, while maintaining stable performance during testing.

The value of the ADI calculated for this workload is equivalent to 5.8034646864472008.

### 4.3 Workload 6

The graph of CPU utilization with workload 6 (peaky) highlights sharp peaks of increasing intensity that progressively stress the system.

After an initial low-use phase, CPU utilization repeatedly reaches values close to 100%, alternating between high-load phases and rapid drops, simulating scenarios of irregular demand. In the final phase, usage decreases until it stabilizes at zero, marking the end of the test. Overall, the system demonstrates a good response to peaks, while showing a variable and nonlinear trend that reflects the pressure exerted by intermittent and short-duration loads.

The graph shows how the desired capacity, that is, the number of machines needed, dynamically varies in response to the applied load.

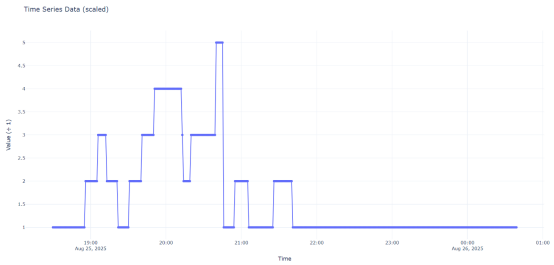


Fig. 6. Peaky desired capacity

After an initial stable phase at 1 machine, the system progressively scales through step increments, reaching up to 5 machines during peak demand moments. The trend is characterized by fluctuations that reflect the variability of the load. Subsequently, the capacity begins to decrease again to the minimum value, indicating the end of the stress phase.

Overall, the auto-scaling mechanism appears effective: it reacts to increases in demand, adapts to peaks, and reduces unnecessary resources, ensuring a good balance between performance and costs.

The value of the ADI calculated for this workload is equivalent to 6.830014450684615.

### 4.4 Analysis of the Results

Performance evaluation experiments show that the adaptive auto-scaling strategy with variable steps is effective in maintaining acceptable service levels in the presence of varying load conditions. The observed behavior is consistent with the theoretical predictions of the paper by Netto et al. (2014) and confirms that the approach can be adopted in real-world scenarios to enhance the elasticity and operational efficiency of cloud infrastructures.

## 5 Conclusions

In this project, an adaptive auto-scaling strategy was implemented for cloud environments, based on the variable step size algorithm proposed by Netto et al. (2014), using AWS services such as Lambda, CloudWatch, Auto Scaling Groups and Application Load Balancer.

The goal was to evaluate whether the system could dynamically adapt to variable workloads while maintaining stable performance, high availability, and efficient resource usage.

The algorithm has been integrated into a Lambda function that regulates the desired capacity of the ASG based on CPU utilization thresholds and a configurable aggressiveness parameter, with CloudWatch, in addition to providing visibility into performance, automating invocations via SNS acting as a control signal generator for adaptive scaling.

The experimental environment was evaluated with three different workload profiles, representative of real scenarios: alternating low activity and sudden spikes (bursty), high activity request groups separated by cooldowns (clustered) and sharp, short-lived demand surges (peaky).

The results obtained from the experiments demonstrate that the solution is able to scale correctly and proportionally, keeping CPUUtilization within range. Furthermore, the adaptive approach reduces the risk of over/under-scaling compared to static strategies or fixed thresholds.

In conclusion, the experiment confirms that the adoption of adaptive scaling techniques on cloud platforms such as AWS can significantly improve the reliability, efficiency, and scalability of systems, representing a valid solution for dynamic environments with high load variability.

## REFERENCES

[1] M. A. Netto, C. Cardonha, R. L. Cunha, and M. D. Assuncao, "Evaluating auto-scaling strategies for cloud computing environments," in *2014 IEEE 22nd International Symposium on Modelling, Analysis Simulation of Computer and Telecommunication Systems*, 2014, pp. 187–196.